

Segurança Informática — Resumo de Matéria

Fonte: Análise cruzada de 5 testes (2019/20 a 2023/24, época normal e recurso). Estrutura ordenada do mais fundamental para o mais complexo.

Índice

1. [Primitivas de Cifra Simétrica em Bloco](#)
 2. [Modos de Operação](#)
 3. [Funções de Hash Criptográficas](#)
 4. [MAC — Message Authentication Code](#)
 5. [Assinatura Digital](#)
 6. [Cifra Autenticada \(AE\)](#)
 7. [Certificados X.509 e PKI](#)
 8. [Protocolo TLS](#)
 9. [JCA — Java Cryptography Architecture](#)
 10. [Armazenamento Seguro de Passwords](#)
 11. [RFC 7516 — JSON Web Encryption \(JWE\)](#)
 12. [Controlo de Acessos e RBAC](#)
 13. [OAuth 2.0](#)
 14. [OpenID Connect](#)
 15. [Bitcoin e Blockchain](#)
 16. [Vulnerabilidades Web — XSS e CSRF](#)
 17. [Termos-Chave](#)
-

1. Primitivas de Cifra Simétrica em Bloco

Conceito

Uma **primitiva de cifra simétrica em bloco** é uma função determinística que transforma um bloco de dados de tamanho fixo usando uma chave secreta. A **mesma chave** é usada para cifrar e decifrar.

- Exemplos: **DES** (Data Encryption Standard), **AES** (Advanced Encryption Standard)
- **Determinismo:** para a mesma chave k e o mesmo bloco b , a função $E(k)(b)$ produz sempre o mesmo resultado.
- A mesma primitiva pode ser usada com **diferentes modos de operação** (ver secção 2).

Chaves Simétricas vs. Assimétricas

- As **chaves simétricas têm menor dimensão** (em bits) do que as chaves assimétricas nos usos modernos típicos. Isto deve-se às diferentes estruturas matemáticas subjacentes.
- Por exemplo, uma chave AES de 128 bits oferece segurança equivalente a uma chave RSA de ~3072 bits.

Padding

Quando a mensagem não é múltipla do tamanho do bloco, é necessário um algoritmo de **padding** para completar o último bloco.

- **Limitação do padding com zeros (0x00):** é ambíguo — não é possível distinguir zeros de padding de zeros que fazem parte da mensagem original. Um esquema correto (ex: PKCS#7) inclui informação sobre quantos bytes foram adicionados.

2. Modos de Operação

Os modos de operação definem como a primitiva em bloco é aplicada a mensagens de tamanho arbitrário.

ECB — Electronic Code Book

- Cada bloco é cifrado **independentemente** com a mesma chave.
- **Não requer IV (vetor inicial).**
- **Inseguro para uso geral:** blocos iguais de plaintext produzem blocos iguais de criptograma, revelando padrões na mensagem.

CBC — Cipher Block Chaining

- Antes de cifrar, cada bloco é sujeito a XOR com o criptograma do bloco anterior.
- **Requer IV** (vetor inicial, aleatório e único por mensagem), usado no primeiro bloco.
- O IV **não precisa de ser secreto**, mas deve ser conhecido pelo destinatário antes da decifra.
- **O modo de cifra e decifra têm de ser consistentes** — não é possível cifrar em ECB e decifrar em CBC com o mesmo criptograma.

GCM — Galois/Counter Mode

- Modo de operação **autenticado**: combina cifra em modo contador (CTR) com autenticação baseada em campo de Galois.
- **Garante confidencialidade e autenticidade** em simultâneo.
- Permite **detetar modificações** no criptograma antes da decifra, sem necessidade de um esquema MAC externo separado.
- Muito usado em protocolos modernos (TLS, JWE).

Resumo comparativo

Modo	Requer IV	Autenticidade	Paralelizável (cifra)
ECB	Não	Não	Sim
CBC	Sim	Não	Não
GCM	Sim	Sim	Sim

3. Funções de Hash Criptográficas

Definição

Uma **função de hash criptográfica** H mapeia uma entrada de tamanho arbitrário para uma saída de tamanho fixo (o *digest* ou *hash*).

Propriedades fundamentais

- **Resistência à pré-imagem:** dado $h = H(x)$, é computacionalmente inviável encontrar x .
- **Resistência à segunda pré-imagem:** dado x , é computacionalmente inviável encontrar $x' \neq x$ tal que $H(x') = H(x)$.
- **Resistência a colisões:** é computacionalmente inviável encontrar qualquer par x, x' com $x \neq x'$ tal que $H(x) = H(x')$.

Exemplos e estado de segurança

- **SHA-1:** Considerado fraco para uso criptográfico moderno (colisões já demonstradas).
- **MD5: Inseguro** — é computacionalmente viável encontrar colisões. Um atacante com acesso a colisões em MD5 pode forjar assinaturas digitais de certificados X.509, substituindo o conteúdo legítimo por um conteúdo malicioso com o mesmo hash.

4. MAC — Message Authentication Code

Definição

Um **MAC** é uma marca (tag) de autenticidade gerada a partir de uma mensagem e de uma **chave secreta partilhada** entre emissor e destinatário.

- Garante **integridade** e **autenticidade** da mensagem.
- **Não garante não-repúdio** — qualquer parte que conhece a chave pode gerar uma marca válida.

Funcionamento geral

```
marca = MAC(k, mensagem)
```

O destinatário verifica recalculando o MAC e comparando com a marca recebida.

Na JCA (Java Cryptography Architecture)

- Classe: **Mac**
- A dimensão da marca depende do algoritmo especificado no método fábrica `getInstance`.
- A classe **Mac** recebe uma **chave secreta** (não um certificado) para gerar a marca.

MAC vs. Assinatura Digital — quando usar cada um?

Critério	MAC	Assinatura Digital
Chaves	Simétrica (partilhada)	Assimétrica (par público/privado)
Não-repúdio	✗ Não	✓ Sim

Critério	MAC	Assinatura Digital
Desempenho	Mais rápido	Mais lento
Verificação por terceiros	✗ Não	✓ Sim
Adequado para cookies de sessão	✓ Sim (servidor conhece chave)	Possível mas desnecessário

Para autenticar cookies de sessão num servidor web, o MAC é mais adequado: o servidor é o único que gera e verifica os cookies, não há necessidade de verificação por terceiros, e é mais eficiente.

5. Assinatura Digital

Definição

Uma **assinatura digital** é um esquema criptográfico assimétrico que garante:

- **Autenticidade:** a mensagem foi produzida pelo detentor da chave privada.
- **Integridade:** a mensagem não foi alterada após a assinatura.
- **Não-repúdio:** o signatário não pode negar ter assinado (apenas ele tem a chave privada).

Processo

- **Assinatura:** usa a **chave privada** do signatário para gerar a assinatura sobre o hash da mensagem.
- **Verificação:** usa a **chave pública** do signatário para verificar a assinatura.

```
assinatura = Sign(chave_privada, H(mensagem))  
Verify(chave_publica, mensagem, assinatura) → verdadeiro/falso
```

Assinatura Digital vs. MAC

A assinatura digital permite verificação por **qualquer entidade** que possua a chave pública, enquanto o MAC requer que o verificador conheça a chave secreta. Assim, **os objetivos de um sistema de assinatura de contratos não poderiam ser igualmente atingidos por um MAC**, pois o MAC não garante não-repúdio (qualquer parte com a chave poderia gerar uma marca) e não permite verificação por terceiros independentes.

6. Cifra Autenticada (AE)

Motivação

A cifra simétrica por si só garante **confidencialidade** mas não **integridade/autenticidade**. Um atacante pode modificar o criptograma sem ser detetado.

Esquema AE (Authenticated Encryption)

O objetivo é combinar cifra com garantia de integridade, de modo a que modificações no criptograma sejam detetadas pelo destinatário.

Exemplo de esquema AE (conceitual):

$$\text{AEe}(k)(m) = E(k)(m) \parallel H(E(k)(m))$$

$$\text{AEd}(k)(c, h) = \text{se } H(c) == h \text{ então } m = D(k)(c) \text{ senão falha de integridade}$$

Limitação deste esquema específico: A integridade é verificada com base no hash do criptograma. Porém, se a função H usada **não for keyed** (não usa chave), um atacante pode modificar tanto c como h de forma consistente, comprometendo a integridade. Um esquema correto deve usar **MAC com chave** ou **GCM** (que integra autenticação no próprio modo de operação).

GCM como AE nativo

O modo **GCM** é hoje a abordagem padrão para cifra autenticada — a autenticidade é garantida internamente, sem necessidade de um MAC externo separado.

7. Certificados X.509 e PKI

O que é um Certificado X.509

Um **certificado X.509** é uma estrutura de dados assinada digitalmente que associa uma **chave pública** a uma identidade (sujeito). Emitido por uma **Autoridade de Certificação (CA)**.

Campos relevantes de um certificado

- **Sujeito (Subject):** identidade do titular (ex: `CN=www.isel.pt`)
- **Chave pública do sujeito**
- **Emissor (Issuer):** CA que assinou o certificado
- **Assinatura digital da CA:** calculada sobre o conteúdo do certificado com a **chave privada da CA**
- **Período de validade**
- **Extensões** (ex: uso de chave, restrições básicas)

⚠ **Nenhum campo do certificado é cifrado** (simétrica ou assimetricamente). A proteção é por **assinatura digital**, não por cifra.

Verificação de um certificado

- A assinatura de um certificado **folha ou intermédio** é verificada com a **chave pública do seu emissor** (não com a chave pública do próprio certificado).
- Dois certificados com o mesmo nome de sujeito podem ter **assinaturas diferentes** (emitidos por CAs diferentes, em momentos diferentes, com chaves privadas diferentes).

Hierarquia de certificados (PKI)

```
CA Raiz (auto-assinada, trust anchor)
├── CA Intermédia (certificada pela CA Raiz)
│   └── Certificado Folha (certificado pelo sujeito final, ex: servidor web)
```

- **Certificado auto-assinado:** emissor == sujeito. **Não é universalmente considerado de confiança** — só é confiável se estiver explicitamente adicionado ao repositório de confiança (TrustStore).
- **A assinatura de um certificado folha não tem em conta toda a cadeia** — apenas o conteúdo do certificado em si é assinado pela CA diretamente superior.

Emissão de certificado — uso de chaves

Quando a CA raiz **R** emite o certificado **F**:

- A CA usa a sua **chave privada** para assinar o certificado F.
- **Não é a chave pública de R que cifra** qualquer parte do certificado — é a chave privada de R que gera a assinatura sobre o hash do conteúdo do certificado F.

TrustStore

- Repositório que armazena **certificados de raiz** (CA raízes) de confiança.
- Usado para autenticar o servidor numa ligação TLS.

8. Protocolo TLS

O **Transport Layer Security (TLS)** é o protocolo padrão para comunicação segura na internet. É composto por dois sub-protocolos principais.

8.1 Handshake Protocol

Estabelece os parâmetros criptográficos da sessão.

Passos principais (TLS 1.2, simplificado):

1. **ClientHello:** cliente envia a lista de cipher suites suportadas e o **client_random**.
2. **ServerHello:** servidor seleciona cipher suite e envia **server_random** e o seu certificado.
3. **Troca de chave:** geração do **pre-master secret** (e derivação das chaves de sessão).
4. **Finished:** ambos enviam mensagens **Finished** com um MAC sobre todo o handshake — **é aqui que ataques de downgrade são detetados**.

Ataque de Downgrade e sua Deteção

Um atacante pode tentar modificar as mensagens iniciais (ClientHello/ServerHello) para forçar o uso de algoritmos mais fracos. **Como é detetado:**

- As mensagens **Finished** contêm um **MAC de todo o handshake** (incluindo as mensagens iniciais). Se estas foram modificadas, o MAC não coincide, e a ligação falha.

client_random e server_random

- São enviados em claro (canal inseguro).
- Se forem modificados pelo atacante, **tal será detetado nas mensagens Finished** do handshake, pelo mesmo mecanismo de MAC sobre o handshake.

Pre-master Secret e Perfect Forward Secrecy (PFS)

- **Abordagem clássica (RSA):** o cliente cifra o pre-master secret com a chave pública do servidor.
Problema: se a chave privada do servidor for comprometida no futuro, um atacante que gravou sessões passadas pode decifrá-las retroativamente.
- **Perfect Forward Secrecy (PFS):** usa troca de chaves efêmeras (ex: Diffie-Hellman Efêmero — DHE/ECDHE). As chaves de sessão são geradas de novo para cada sessão e **não dependem da chave privada de longo prazo do servidor**. Mesmo que a chave privada do servidor seja comprometida, as sessões passadas permanecem seguras.
 - O RFC 7525 classifica como insegura a troca do pre-master secret via cifra com chave pública (RSA estático) precisamente por não garantir PFS.
 - **Objetivo da PFS:** proteger contra ataques ao **servidor** (comprometimento da chave privada de longo prazo).

8.2 Record Protocol

Responsável pela **transmissão segura dos dados da aplicação** após o handshake.

- Garante **confidencialidade** e **integridade/autenticidade** dos dados.
- As chaves usadas são **chaves simétricas de sessão** derivadas do handshake — **não são as chaves privadas do servidor**.
- **A integridade no record protocol é garantida com MAC** (esquema simétrico), não com assinatura digital assimétrica.

TLS e HTTPS

Em HTTPS, o TLS encapsula o HTTP. O **record protocol** é responsável pela confidencialidade e autenticidade dos cookies nas trocas HTTP.

Man-in-the-Middle via CA Comprometida

Um ataque a uma CA **diferente da CA0** (que emitiu o certificado do servidor S) pode comprometer uma ligação:

- Se o atacante comprometer outra CA, pode emitir um certificado fraudulento para o servidor S.
- O cliente C confia em múltiplas CAs raiz (via TrustStore do sistema operativo/browser). Se qualquer uma delas for comprometida, o atacante pode realizar um MITM, apresentando o certificado fraudulento.

9. JCA — Java Cryptography Architecture

A **JCA** é uma framework Java para operações criptográficas, que segue o princípio de **independência de algoritmos**: a API é independente dos algoritmos concretos que implementam cada esquema.

Princípio de Independência de Algoritmos

Exemplo: a classe `Cipher` pode ser usada com AES, DES ou qualquer outro algoritmo, sem alterar o código da aplicação — apenas o parâmetro passado ao método fábrica `getInstance` muda.

```
Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");  
// ou
```

```
Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
```

Classe `Cipher`

Principal classe para operações de cifra/decifra.

Método `init`: inicializa a cifra com o modo de operação:

Modo	Descrição
<code>ENCRYPT_MODE</code>	Configura para cifrar — recebe a chave (e IV se necessário) e produz criptograma
<code>DECRYPT_MODE</code>	Configura para decifrar — recebe a chave e produz plaintext
<code>WRAP_MODE</code>	Cifra uma chave criptográfica (key wrapping) para transporte seguro
<code>UNWRAP_MODE</code>	Decifra uma chave criptográfica previamente wrapped

Métodos `update` e `doFinal`:

- `update(bytes)`: processa dados parciais (útil para streams de dados grandes). Pode ou não produzir output imediatamente, dependendo do algoritmo/modo.
- `doFinal(bytes)`: sinaliza o **fim** dos dados, processa os últimos bytes, aplica padding (se necessário) e **liberta recursos internos** (ex: flush de buffers internos, finalização do MAC em GCM). Sem `doFinal`, a cifra/decifra pode estar incompleta.

Classe `Mac`

- Implementa esquemas MAC.
- Recebe uma **chave secreta** (não um certificado) para gerar a marca.
- A dimensão da marca depende do algoritmo indicado em `getInstance`.

Classe `X509Certificate`

- Método `getPublicKey()` — obtém a chave pública do certificado. ✓
- **Não existe** método `getPrivateKey()` — os certificados X.509 **não contêm chaves privadas**. ✗

10. Armazenamento Seguro de Passwords

Problema

Guardar passwords em claro ou cifradas com chave simétrica é perigoso: se a base de dados for comprometida, todas as passwords (e a chave) ficam expostas.

Solução com Hash + Salt

Armazenar a informação de validação na forma:

```
v_u = H(pwd_u || s_u)
```

onde s_u é um **salt aleatório e único por utilizador**, e H é uma função de hash criptográfica (idealmente uma função de hash para passwords, como bcrypt, scrypt ou Argon2).

Vantagens face à cifra simétrica:

- Mesmo que a base de dados seja comprometida, o atacante não obtém as passwords diretamente (a função de hash é one-way).
- Com cifra simétrica, se a chave k for também armazenada na base de dados, o atacante obtém tudo.

O papel do Salt

- **Sem salt:** passwords iguais produzem o mesmo hash. Um atacante pode usar **tabelas de pré-computação (rainbow tables)** para reverter hashes.
- **Com salt único por utilizador:** mesmo que dois utilizadores tenham a mesma password, os hashes são diferentes. As rainbow tables tornam-se ineficazes.
- **Tamanho do salt e ataques pela interface de autenticação:** aumentar o salt de 16 para 64 bits **não aumenta a dificuldade de ataques feitos através da interface de login** (tentativa por tentativa). O salt maior apenas dificulta ataques offline de pré-computação. Ataques pela interface são limitados por outras medidas (rate limiting, lockout).

Ataque a um esquema fraco

Esquema vulnerável: $v_u = \text{SHA1}(\text{pwd}_u \parallel \text{SHA1}(\text{pwd}_u)_{\{1..32\}})$

- O valor $\text{SHA1}(\text{pwd}_u)_{\{1..32\}}$ (primeiros 32 bits do hash) tem apenas $2^{32} \approx 4 \text{ mil milhões}$ de valores possíveis — muito reduzido.
- Um atacante pode computar $\text{SHA1}(\text{pwd})_{\{1..32\}}$ para um dicionário de passwords comuns e depois calcular o v_u correspondente, comparando com os valores armazenados. O espaço de busca efetivo é muito menor do que parece.

11. RFC 7516 — JSON Web Encryption (JWE)

Motivação para Duas Chaves

O RFC 7516 usa **duas chaves distintas** no processo de cifra:

- **CEK (Content Encryption Key):** chave simétrica gerada aleatoriamente por sessão, usada para cifrar o conteúdo (texto t) com um algoritmo simétrico (ex: AES-GCM).
- **Chave pública do destinatário (Ke):** usada para cifrar a CEK (key wrapping).

Razão: A cifra assimétrica (usada para proteger a CEK) é **muito mais lenta** e tem **limitações de tamanho** — não é adequada para cifrar mensagens longas diretamente. A cifra simétrica (usada para o conteúdo) é eficiente. Esta abordagem híbrida combina o melhor dos dois mundos.

Processo de Decifra

1. O destinatário usa a sua **chave privada** para decifrar a CEK (key unwrapping).
2. Com a CEK recuperada, decifra o texto t usando o algoritmo simétrico.

Autenticidade em JWE com GCM

Quando se usa o modo **GCM** para cifrar o texto, a autenticidade **já está integrada** no próprio modo de operação — o GCM produz uma authentication tag. **Não é necessário um esquema MAC externo adicional**, ao contrário do que sucederia com modos de operação não autenticados (ex: CBC).

12. Controlo de Acessos e RBAC

Conceitos Gerais

- **Autenticação:** verificar a identidade de um utilizador.
- **Autorização (controlo de acessos):** verificar se o utilizador autenticado tem permissão para realizar uma ação.
- **Ordem:** a autenticação **precede** o controlo de acessos — não faz sentido autorizar antes de saber quem é o utilizador.

Modelos de Controlo de Acessos

- **ACL (Access Control List):** lista de utilizadores/grupos com as suas permissões por objeto. Vantagem: simplifica a listagem das permissões associadas a um objeto específico.
- **RBAC (Role-Based Access Control):** permissões associadas a **papéis (roles)**, e utilizadores atribuídos a papéis.

RBAC — Modelo Base

Componentes:

- **U** — conjunto de utilizadores
- **R** — conjunto de papéis (roles)
- **P** — conjunto de permissões
- **UA (User Assignment):** relação $U \times R$ — que papéis estão atribuídos a cada utilizador.
- **PA (Permission Assignment):** relação $R \times P$ — que permissões estão atribuídas a cada papel.

⚠ **PA relaciona roles e permissões** — não relaciona utilizadores diretamente. O RBAC base considera apenas **permissões positivas** (o que é permitido; o que não está listado é negado).

RBAC1 — Hierarquia de Roles

O **RBAC1** adiciona uma **hierarquia de roles (RH)**, onde $r1 \leq r2$ significa que $r2$ é hierarquicamente superior a $r1$ e **herda todas as permissões de $r1$** .

Exemplo:

$$RH = \{r0 \leq r1, r0 \leq r2, r2 \leq r3, r1 \leq r4\}$$

Isto significa:

- $r1$ herda as permissões de $r0$
- $r2$ herda as permissões de $r0$
- $r3$ herda as permissões de $r2$ (e transitivamente de $r0$)

Consulta de permissões de uma sessão: numa sessão s com $user(s) = u$, os roles ativos $roles(s)$ são um subconjunto dos roles atribuídos ao utilizador u e dos roles inferiores na hierarquia (roles que o utilizador pode "ativar").

Sessões em RBAC

A **sessão** é um conceito distinto da relação UA. Motivação:

- A relação UA apenas define **quais papéis um utilizador pode assumir** — não qual está ativo em cada momento.
- Uma sessão representa uma interação ativa do utilizador, onde apenas **um subconjunto** dos papéis disponíveis está ativo.
- Permite implementar o **princípio do mínimo privilégio**: o utilizador ativa apenas os papéis necessários para a tarefa corrente.
- Exemplo: um professor no Moodle pode ter os papéis "professor", "professor não editor", "aluno" ou "visitante" — mas apenas **um** pode estar ativo por sessão.

Casbin

Biblioteca para controlo de acessos em aplicações (não exclusivamente web) — implementa vários modelos, incluindo RBAC.

13. OAuth 2.0

Objetivo

OAuth 2.0 é uma framework de **autorização** que permite que uma aplicação cliente aceda a recursos protegidos em nome de um utilizador (dono de recursos), sem que a aplicação obtenha as credenciais do utilizador.

Entidades

- **Resource Owner (Dono de Recursos):** utilizador final que autoriza o acesso.
- **Client (Aplicação Cliente):** aplicação que solicita acesso. **Não é o browser.**
- **Authorization Server (Servidor de Autorização):** emite tokens após autenticação e autorização.
- **Resource Server (Servidor de Recursos):** onde os recursos protegidos residem.

Fluxo Authorization Code Grant

1. Aplicação cliente redireciona o browser para o Authorization Endpoint (com: `client_id`, `redirect_uri`, `scope`, `state`, `response_type=code`)
2. Utilizador autentica-se e autoriza o acesso no Authorization Server.
3. Authorization Server redireciona o browser para o callback (`redirect_uri`) da aplicação cliente, com o authorization code (e state).
4. Aplicação cliente usa o authorization code para contactar diretamente o Token Endpoint (back-channel), obtendo o `access_token`.

5. Aplicação cliente usa o `access_token` para aceder ao Resource Server.

Parâmetros Importantes

- **client_id** e **client_secret**: identificador e segredo da aplicação cliente, obtidos durante o **registo prévio** da aplicação no servidor de autorização.
- **scope**: define o âmbito do acesso solicitado. O valor é **escolhido pela aplicação cliente** (não pelo dono de recursos), com base nos recursos a que precisa aceder.
- **access_token**: token opaco (para o cliente) que autoriza o acesso ao Resource Server. **Não é destinado a ser interpretado pela aplicação cliente** — é enviado ao Resource Server.
- **state**: parâmetro aleatório e imprevisível gerado pela aplicação cliente, enviado no pedido de autorização e devolvido na resposta. Serve para **proteção contra ataques CSRF** — a aplicação cliente deve verificar que o **state** recebido na resposta corresponde ao enviado.

Callback (redirect_uri)

- Refere-se a um **endpoint da aplicação cliente** (não do servidor de autorização).
- A comunicação entre a aplicação cliente e o Resource Server **nunca usa o callback**.

14. OpenID Connect

Objetivo

OpenID Connect (OIDC) é uma camada de **autenticação** construída sobre o OAuth 2.0. Enquanto o OAuth 2.0 trata de **autorização**, o OIDC permite verificar a identidade do utilizador.

Diferença face ao OAuth 2.0

	OAuth 2.0	OpenID Connect
Objetivo	Autorização	Autenticação
Token principal	<code>access_token</code>	<code>id_token</code> (+ <code>access_token</code>)
<code>scope</code> obrigatório	Específico do serviço	Inclui <code>openid</code>

`id_token` vs. `access_token`

- **access_token**: usado para aceder ao **Resource Server** em nome do utilizador. Opaco para a aplicação cliente.
- **id_token**: token JWT que contém **informação de identidade sobre o utilizador** (sub, name, email, etc.), destinado a ser lido pela **aplicação cliente** para confirmar quem é o utilizador.

UserInfo Endpoint

Para obter informação adicional sobre o utilizador (ex: foto de perfil), a aplicação cliente acede ao **UserInfo Endpoint** do servidor de autorização, apresentando o `access_token`.

Parâmetro **state** e Segurança

O parâmetro **state** em OIDC tem a mesma função que em OAuth 2.0 — **proteção CSRF**. A aplicação cliente deve:

1. Gerar um valor aleatório e imprevisível antes de redirecionar.
2. Armazená-lo (ex: em sessão).
3. Na resposta, verificar que o **state** recebido corresponde ao enviado — caso contrário, rejeitar.

Ataque de Session Fixation em OIDC

Um atacante pode iniciar o fluxo de autenticação numa aplicação, interromper o processo quando a resposta passa pelo seu browser (obtendo o authorization code), e depois tentar que a vítima complete o fluxo com esse code — autenticando a vítima como o atacante. **Mitigação:** o parâmetro **state** ligado à sessão do browser impede este ataque, pois o **state** do atacante não corresponde à sessão da vítima.

15. Bitcoin e Blockchain [ORDEM INCERTA]

Estrutura de um Bloco

No Bitcoin, as transações são organizadas em **blocos encadeados**. Cada bloco contém um **header** com:

- **h1** = **H(header do bloco anterior)** — hash do bloco precedente (garante encadeamento)
- **h2** = **H(transações do bloco atual)** — hash da Merkle tree das transações (garante integridade das transações do bloco)

Integridade da Cadeia

A propriedade de **resistência a colisões** das funções de hash criptográficas garante que:

- Para alterar as transações de um bloco **bi**, seria necessário alterar **h2** no header de **bi**.
- Ao alterar **h2**, o hash do bloco **bi** muda — o que invalida **h1** no bloco seguinte **bi+1**.
- Seria necessário recalcular toda a cadeia a partir de **bi** — computacionalmente inviável numa rede com prova de trabalho.

Conclusão: alterar transações de um bloco sem alterar nenhum outro **não é computacionalmente viável**, dado que uma boa função de hash criptográfica torna inviável encontrar uma segunda pré-imagem que produza o mesmo **h2**.

16. Vulnerabilidades Web — XSS e CSRF [ORDEM INCERTA]

XSS — Cross-Site Scripting

Definição: o atacante injeta código JavaScript malicioso numa página web, que é executado no browser da vítima.

Condições para XSS num cliente de e-mail:

- O cliente de e-mail tem de **renderizar HTML** (não apenas texto plano).

- Se o conteúdo HTML do email não for sanitizado antes de ser renderizado, código JavaScript no corpo do email pode ser executado no contexto do cliente.

CSRF — Cross-Site Request Forgery

Definição: o atacante engana a vítima para que o browser da vítima faça um pedido indesejado a uma aplicação onde a vítima está autenticada.

No contexto de OAuth 2.0 / OIDC: o parâmetro **state** protege contra CSRF no fluxo de autorização — impede que um atacante substitua o authorization code legítimo por um seu.

Validação do **state**:

- A aplicação cliente deve associar o **state** à sessão de browser antes de redirecionar.
- Na resposta, verificar que o **state** recebido corresponde exatamente ao gerado e armazenado para essa sessão.

Termos-Chave

Termo	Definição Breve
AES	Advanced Encryption Standard — primitiva de cifra simétrica em bloco
DES	Data Encryption Standard — primitiva de cifra simétrica em bloco (obsoleto)
ECB	Electronic Code Book — modo de operação sem IV, inseguro para dados com padrões
CBC	Cipher Block Chaining — modo de operação com XOR encadeado e IV
GCM	Galois/Counter Mode — modo de operação autenticado (confidencialidade + integridade)
Padding	Preenchimento do último bloco para múltiplo do tamanho de bloco
IV	Vetor Inicial — valor aleatório para tornar a cifra não-determinística por mensagem
Hash criptográfico	Função one-way, tamanho fixo de output, resistente a colisões
SHA-1	Secure Hash Algorithm 1 — considerado fraco (colisões demonstradas)
MD5	Message Digest 5 — inseguro, colisões computacionalmente viáveis
MAC	Message Authentication Code — marca de autenticidade com chave simétrica partilhada
Assinatura Digital	Esquema assimétrico com não-repúdio — assina com chave privada, verifica com pública
AE	Authenticated Encryption — combinação de cifra + autenticidade
Cifra Híbrida	Combina cifra assimétrica (para chave) + simétrica (para dados)
X.509	Norma para certificados de chave pública

Termo	Definição Breve
CA	Certification Authority — autoridade que emite e assina certificados
PKI	Public Key Infrastructure — infraestrutura de gestão de certificados e CAs
TrustStore	Repositório de certificados raiz de confiança
Certificado folha	Certificado de entidade final (ex: servidor web), não emite outros certificados
TLS	Transport Layer Security — protocolo de comunicação segura
Handshake Protocol	Sub-protocolo TLS para negociação de parâmetros e autenticação
Record Protocol	Sub-protocolo TLS para transmissão segura de dados da aplicação
PFS	Perfect Forward Secrecy — garante que sessões passadas são seguras mesmo com chave privada comprometida
DHE/ECDHE	Diffie-Hellman Efêmero — mecanismo de troca de chaves que garante PFS
Downgrade attack	Ataque que força uso de algoritmos criptográficos mais fracos
JCA	Java Cryptography Architecture — API Java independente de algoritmos
WRAP/UNWRAP	Modos da classe Cipher para cifrar/decifrar chaves criptográficas
Salt	Valor aleatório único por utilizador adicionado à password antes de fazer hash
Rainbow Table	Tabela pré-computada de hashes para reverter passwords — mitigada pelo salt
JWE	JSON Web Encryption (RFC 7516) — norma para cifra de dados em formato JSON
CEK	Content Encryption Key — chave simétrica efêmera usada em JWE
RBAC	Role-Based Access Control — controlo de acessos baseado em papéis
RBAC1	RBAC com hierarquia de roles (RH)
UA	User Assignment — relação utilizadores ↔ roles em RBAC
PA	Permission Assignment — relação roles ↔ permissões em RBAC
Sessão RBAC	Instância ativa de interação com subset de roles disponíveis
ACL	Access Control List — lista de permissões por objeto
Casbin	Biblioteca de controlo de acessos para múltiplos modelos (incluindo RBAC)
OAuth 2.0	Framework de autorização delegada
OIDC	OpenID Connect — camada de autenticação sobre OAuth 2.0
access_token	Token OAuth para aceder ao Resource Server
id_token	Token OIDC com informação de identidade do utilizador (JWT)
Authorization Code Grant	Fluxo OAuth/OIDC com código de autorização intermediário

Termo	Definição Breve
scope	Parâmetro que define o âmbito do acesso solicitado
state	Parâmetro anti-CSRF nos fluxos OAuth/OIDC
callback (redirect_uri)	Endpoint da aplicação cliente para receber respostas do servidor de autorização
Blockchain	Cadeia de blocos encadeados por hashes — integridade garantida por hash criptográfico
XSS	Cross-Site Scripting — injeção de código JavaScript malicioso
CSRF	Cross-Site Request Forgery — pedido forjado em nome do utilizador autenticado